DS&AI Lab Project [Term Jan 2026]

Action-Oriented Indoor Navigation Assistance for the Visually Impaired

MILESTONE-6: The Final Project Report

Indian Institute of Technology Madras

GROUP: 4

Name	Email	GitHub Usernames
Saransh Saini	22f1001123@ds.study.iitm.ac.in	Saransh482003
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	22f1000411
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	SamyukthaSh24
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	23f3003434
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	rohitblpprajapat

1. Introduction and Problem Statement Definition

1.1. Background and Motivation

Visual impairment is a significant global health challenge that profoundly impacts individual autonomy and quality of life. As of 2024, the World Health Organization (WHO) estimates that at least **2.2 billion people** globally have a near or distance vision impairment. In India alone, there are approximately **62 million visually impaired persons**, including **8 million who are completely blind**, representing nearly a quarter of the global burden. Projections indicate these numbers will rise significantly by 2050 due to aging populations and the increasing prevalence of conditions like diabetic retinopathy.

While outdoor navigation has been revolutionized by Global Positioning Systems (GPS) and accessible mapping, indoor environments remain a "black hole" for assistive technology. Indoor spaces are characterized by:

- **Infrastructure Limitations:** GPS signals are attenuated by walls and ceilings, rendering them ineffective for precise positioning.
- **Dynamic Complexity:** Frequent changes in furniture layouts, the presence of moving people, and temporary obstructions (e.g., a bag left on a corridor floor) create a hazardous environment.
- **Near-Field Risks:** Research indicates that the white cane, while foundational, often fails to detect head-level obstacles or low-profile hazards like slippery floors, leading to a high risk of physical injury.

1.2. The Problem: Descriptive vs. Action-Oriented Assistance

Existing assistive devices primarily focus on **descriptive awareness**—announcing "Chair at 2 o'clock." However, for a user in motion, this information is insufficient. Knowing an object exists does not inherently inform the user whether their current trajectory is safe or if they are about to collide.

Our project addresses this "interpretation gap." The problem we solve is the transition from passive scene understanding to **active, action-oriented guidance**. We focus on the critical near-field range (0–3 meters) where immediate movement decisions must be made to ensure safe, uninterrupted walking.

2. Existing Technologies and Research Gaps

2.1. Review of Current Solutions

The field of indoor navigation for the visually impaired is currently divided into two primary technological categories:

1. **Sensory-Based Infrastructure:**
    - **RFID and BLE Beacons:** These systems utilize radio-frequency identification or Bluetooth Low Energy tags placed throughout a building to localize a user. While accurate (often within 0.15m), they require significant investment in hardware infrastructure and are not scalable across diverse, unmapped indoor environments.
    - **WiFi Trilateration:** Uses existing internet access points for positioning. While cost-effective, it often lacks the precision (often >1m error) required for safe obstacle avoidance.

2. **Vision-Based Systems:**
    - **Object Identification Apps:** Tools like "Seeing AI" or "TapTapSee" use smartphone cameras to identify objects. However, they are often cloud-dependent, introducing significant latency that makes them unusable for real-time navigation.
    - **Lidar and Stereo Cameras:** High-end wearables use depth-sensing hardware to map environments. While effective, these devices are often heavy, expensive, and visually stigmatizing for the user.

2.2. Identified Gaps

Through our literature review in Milestone 1 and initial testing in Milestone 3, we identified three critical gaps:

- **The Latency Bottleneck:** Many systems involve a 500ms+ delay between detection and feedback. In a walking scenario, a 200ms delay can be the difference between stopping safely and hitting an obstacle.
- **Infrastructure Dependency:** Most high-precision systems require the environment to be "prepared" with tags or beacons, limiting the user's freedom to navigate new spaces.
- **Logic Hallucination:** Recent attempts to use Large Language Models (LLMs) for navigation instructions, while natural-sounding, introduce the risk of "hallucinations"—where the model suggests a path that contradicts the raw sensor data.

2.3. Our Proposed Solution: The Deterministic Pipeline

Our project bridges these gaps by implementing a **local, smartphone-based pipeline** that requires no external infrastructure.

- **Action-Oriented Logic:** Instead of just naming objects, our system uses a ***Rule-Based Risk Assessment Module***. By calculating the overlap between detected objects and a predefined "Walking Zone," it translates complex visual data into simple, life-saving commands like "Stop" or "Move Left."
- **Offline Efficiency:** By utilizing quantized models (YOLO11s and Depth-Anything-V2-Metric-Hypersim-VITS), we ensure the system runs locally on-device, preserving user privacy and eliminating internet-related latency.

3. The Project Architecture

The architecture of our active blind navigation assistance system is designed as a low-latency, dual-stream sensor fusion pipeline. Built for edge-device deployment, it completely avoids reliance on cloud computation APIs to ensure user privacy, continuous availability, and near-zero latency.

The system continuously captures raw visual feeds, interprets them simultaneously through semantic (object detection) and geometric (depth estimation) neural pathways, maps these insights onto spatial "zones," and generates non-intrusive Text-to-Speech (TTS) commands.

3.1. The Perception Layer (Dual-Stream Setup)

At the heart of the system is the Perception Layer, which processes each raw RGB frame. To guarantee physical safety, we split perception into two distinct streams: Semantic Representation and Geometric Depth Estimation.

1. **Stream A: Semantic Object Detection**
    - **Core Technology:** A robust and lightweight Ultralytics YOLO11s model. To optimize for high-speed edge hardware, the 'Small' (s) variant of YOLO11 was chosen for its ideal balance between inference latency and AP (Average Precision).
    - **Operation:** When an RGB frame enters this module, YOLO11s scans for 11 specific indoor navigational hazard classes (e.g., person, table, sofa, bed, door, refrigerator, etc.).
    - **Output:** The model outputs a set of bounding boxes with semantic class labels. These parameters contextualize what the obstacle is, feeding the navigational logic with qualitative awareness.

2. **Stream B: Geometric Depth Estimation**
    - **Operation:** Unlike YOLO11s, which only looks for known learned patterns, the monocular depth stream maps the absolute geometry of the environment by calculating the proximity of every single pixel to the camera lens.
    - **Output:** A dense, continuous depth map where pixel intensities correlate directly to real-world distances (in meters). This represents the unarguable physical truth of the room. Unrecognized objects or irregularly shaped furniture will still register as a physical blockade on this map.

3.2. Navigation Logic & Sensor Fusion Layer

This component operates as the deterministic "brain" of the application. The frame is visually sliced into three operational Spatial Zones:

- **Left Zone:** The left 30% of the active frame.
- **Center Zone:** The middle 40% of the active frame (representing the primary trajectory path).
- **Right Zone:** The right 30% of the active frame.

The Sensor Fusion Principle (How Layers Interact):
A critical design requirement is how the pipeline handles conflicts between YOLO11s and Depth-Anything-V2. Bounding boxes alone are geometrically unreliable—for instance, a YOLO bounding box might frame a distant wardrobe with the same physical screen area as a close-up chair.

To resolve this safely, the system computes $danger\_ratios$ and $warning\_ratios$ by mapping YOLO bounding box intersections over the depth maps layout, but strictly prioritizes Depth Estimation.

- The Depth hazard danger weight is mathematically forced to a massively higher scaling factor ($depth\_hazard\_danger\_weight = 25.0$) than default object-detection triggers ($max\_object\_risk = 8.0$).
- Summary of interaction: YOLO11s provides the semantic subject ("Chair detected"), but the Dense Depth Engine acts as the definitive safety veto constraint. Depth determines when the obstacle is dangerous enough to fire a command.

3.3. The Decision & Smoothing Engine

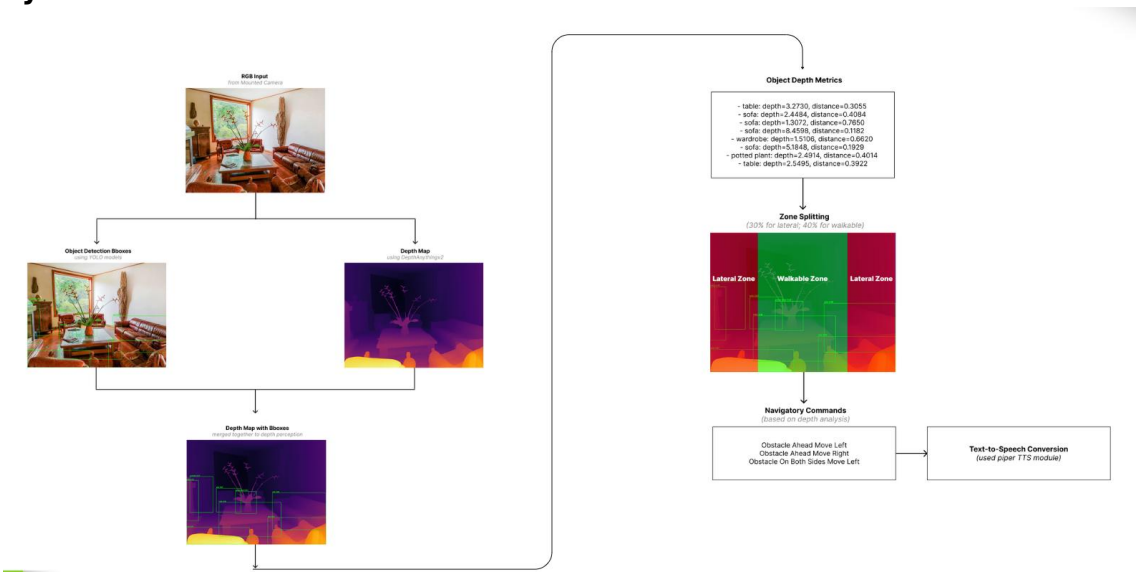
Once spatial risk scores are assigned to the Left, Center, and Right zones, the variables pass through temporal smoothing to prevent erratic behavior:

- **Exponential Moving Average (EMA):** Applied with an ema_alpha = 0.3 to act as a low-pass filter, preventing a single corrupted frame camera-glitch from triggering false alarms.
- **Hysteresis Loop:** If the system is actively commanding the user to *"Turn left,"* it imposes a mathematical bias (turn_hysteresis = 0.5). The alternate path must become definitively clear before the engine allows a contradictory command, ensuring smooth turns.
- **Event Locks:** An internal timer ensures auditory commands stay locked in execution long enough to act on them before the next loop overrides them.

3.4. Execution & Application Layer

- **Text-to-Speech (TTS) Controller:** Commands derived from the logic engine (e.g., "Path blocked, turn left") are routed to the Piper TTS engine (en_US-amy-medium). Using ONNX Runtime, Piper synthesizes the 22.05 kHz WAV audio purely in RAM (eliminating disk I/O chokepoints). It operates on an Async Event Queue, consistently generating speech faster than real-time playback (RTF > 0.3).
- **Performance Diagnostics Pipeline:** A diagnostic wrapper tracks microsecond-accurate lifecycle milestones (YOLO latency, Depth latency, TTS synthesis time) to ensure the system consistently clears real-time inference hurdles without dropping frames.

3.5. System Flowchart



4. Dataset Collection & Preprocessing

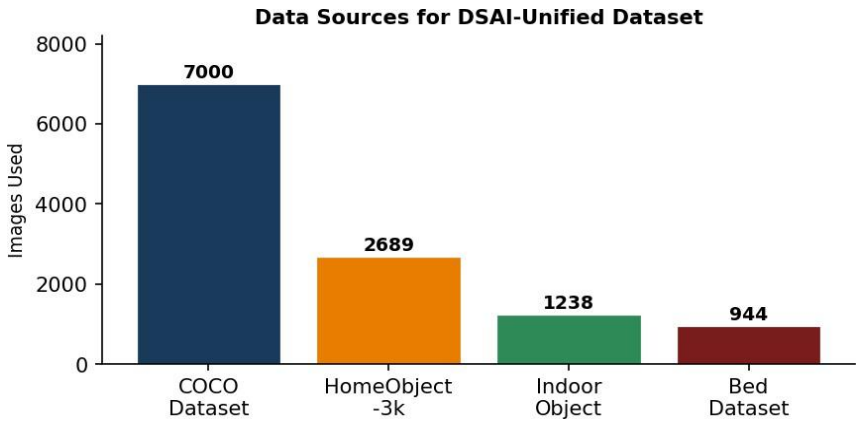
To push the YOLO11s model beyond standard general-purpose benchmarks, we synthesized the dsai-unified-dataset by aggregating and filtering public datasets to over-represent specific, highly dangerous indoor obstacles.

4.1. Object Detection Datasets & Processing

Our team concentrated exclusively on 11 core classes typical to indoor navigation: **person, chair, table, sofa, bed, wardrobe, door, potted plant, lamp, refrigerator,** and **cabinet**.

- **COCO Dataset Subsets:**
  - **Core Extraction Data:** Python pipelines iteratively combed through the standard COCO annotations to extract basic household hazards.
  - **Class Augmentation:** Upon discovering a severe class mismatch that underrepresented large indoor appliances and furniture, individual sub-teams utilized specific automated extraction notebooks (coco-bed.ipynb and coco-refregerator.ipynb) to mine, validate, and standardize heavy obstacles like beds and refrigerators to patch the dataset imbalance.
- **HomeObjects-3K:**
  - Adding further generalization and bounding box refinement, HomeObjects-3K samples were normalized into the YOLO directory constraints to buffer standalone datasets.

Dataset Name	HomeObject-3k	DSAI-Unified	DSAI-Uni-3Cls
Train Samples	2285	9358	9358
Test Samples	404	2340	2340
# of Classes	12	11	3
# of Models Trained	3	9	2
Best Model	YOLO11n	YOLO11s	YOLOv8n
Link to Dataset	HomeObjects-3K	DSAI Unified	DSAI Unified

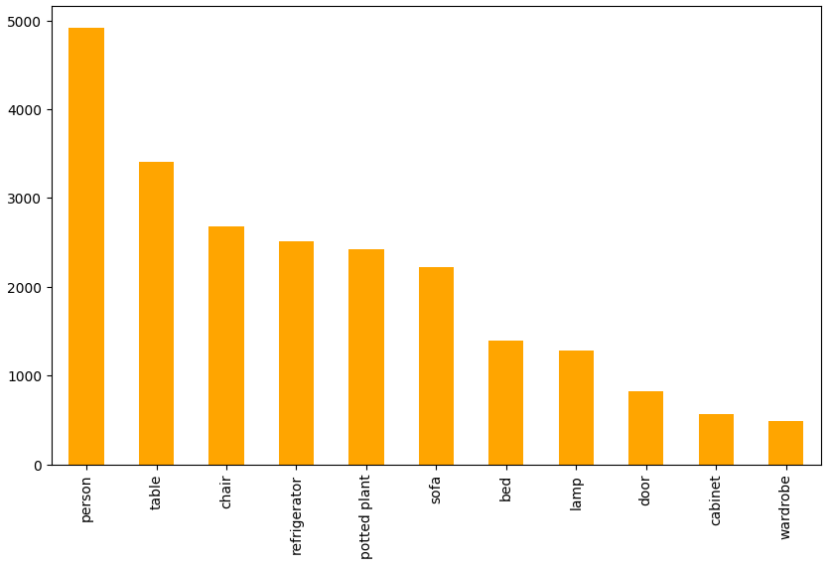


Final Unified Dataset Advantages:

By merging the datasets, the final curated class distribution proved remarkably stable for YOLO11s training:

- Person: 4,174 instances
- Table: 2,920 instances
- Refrigerator: 2,417 instances
- Potted Plant: 2,180 instances

- Chair/Sofa: >4,000 instances
- Bed: 1,407 instances



Why this combination excels: Pooling general-purpose COCO parameters inside a combined dataset of COCO, HomeObject3k and Indoor Image Dataset means the model trains specifically for the complex, low-visibility interactions visually impaired users face daily.

Preprocessing Steps

Before feeding raw imagery into the model, the following preprocessing pipeline is implemented to ensure data consistency and improve generalization:

- **Resizing:** All input images are resized to a fixed resolution of 640×640 pixels to match the network's input layer.
- **Normalization:** Pixel values are scaled from the range $[0,255]$ to $[0,1]$.
- **Data Augmentation:** To prevent overfitting and improve performance in diverse indoor lighting/angles, the following augmentations were applied during training:
    - **Mosaic Augmentation:** Combining four images into one to help the model learn to detect objects at different scales and positions.
    - **Geometric Transforms:** Rotation ($\pm 8.0°$), translation (± 0.08), and scaling (± 0.25).
    - **Color Jittering:** Randomly adjusting Hue (0.015), Saturation (0.7), and Value (0.4).
    - **Flipping:** Horizontal flipping with a 50% probability.

4.2. Depth Estimation Dataset

The depth estimation module was evaluated using both benchmark data and system-level inputs to ensure correctness as well as real-world applicability.
For quantitative evaluation, the NYU Depth V2 dataset was used. This dataset consists of indoor RGB images paired with ground truth depth maps, and is widely used for benchmarking monocular depth estimation models. Evaluation was performed on a held-out subset consistent with standard benchmark usage, and no additional training or fine-tuning was performed on this dataset.
For system-level evaluation, two sources were used. First, a set of 60 images from the SUNRGBD dataset was passed through the full pipeline. Second, structured real-world testing was performed using a live webcam in indoor environments, covering scenarios such as open paths, cluttered spaces, low-texture surfaces, and reflective conditions.
Input images are directly passed to the object detection and depth estimation modules without explicit manual preprocessing. The depth model (DA v2 small) internally performs the required resizing and normalization as part of its pretrained pipeline. The predicted depth map is then resized to match the original image dimensions for downstream processing. No data augmentation is applied during evaluation, as the system operates purely in inference mode.

4.3. Text-to-Speech (TTS) Dataset Collection & Strategy

Unlike our vision models which required massive image datasets for training, our Text-to-Speech (TTS) strategy focused on rigorous dataset collection for latency benchmarking and stress testing. Because our objective prioritizes action-oriented navigation (e.g., “Move left”) over descriptive awareness (e.g., “There is a chair”), we needed text datasets that proved our standalone Piper TTS engine (en_US-amy-medium ONNX model) could guarantee near-instantaneous auditory synthesis without bottlenecking the visual pipeline.

To evaluate the TTS implementation under both realistic navigation scenarios and artificial stress conditions, we curated a comprehensive text benchmark dataset totalling 165 samples across four distinct categories.

1. **Custom Navigation Commands (Real-time Usage)**
    - **Collection:** We manually curated 15 critical, action-oriented directives specifically tailored to the blind-navigation domain (e.g., "Path blocked, turn left", "Obstacle ahead, move right", "Safe to proceed").
    - **Purpose:** To measure the absolute base latency for the exact phrases the system outputs in production.
    - **Characteristics:** These strings are short (averaging 23.9 characters) but command execution is critical. This dataset proves the system can synthesize critical hazard warnings in roughly ~500-800 ms with an excellent Average Real-Time Factor (RTF) of 0.533 (excluding the initial cold-start).

2. **CMU Arctic Dataset (Phonetic Balance)**
    - **Collection:** 50 samples randomly extracted from the 1,132 total CMU Arctic sentences.
    - **Purpose:** To test the engine's ability to handle phonetically balanced sentences (averaging 46.8 characters) without stuttering or dropping phonetic accuracy during synthesis.
    - **Performance:** Yielded a highly reliable RTF ranging strictly between 0.242 and 0.877 (Average: 0.372).

3. **LJ Speech Dataset (Natural Sentences)**
    - **Collection:** 50 samples randomly sampled from the 13,100 total LJ Speech entries.
    - **Purpose:** To evaluate the system on fluid, natural, conversational phrasing (averaging 99.8 characters) predicting how the model handles slight deviations from standard commands.

4. **LibriSpeech Test-Clean (Long Text Stress Test)**
    - **Collection:** 50 samples randomly sampled from a pool of 2,620 total LibriSpeech instances.
    - **Purpose:** Used as a pure stress test. These samples contain long, complex strings (averaging 123.7 characters).
    - **Outcome:** Even at ~4.7x the length of our navigation commands, the average RTF remained heavily optimized at 0.292, proving that TTS latency scales sub-linearly with text length.

Data Preparation & Synthesis Simulation Environment:
Our dataset testing approach was strict regarding production simulation. The text data was not written to disk; instead, the evaluation pipeline parsed the strings directly into WAV (22.05 kHz) bytes strictly within system RAM using an Async Event queue. We intentionally captured the "cold-start" (no warm-up phase) on the very first sample to observe real-world initial latency (which spiked the RTF to 3.630). By profiling the datasets in this manner, we mathematically deduced that the TTS engine processes text ~3x faster than real-time playback (Overall Avg RTF: 0.346 across all 165 inputs), keeping integration overhead to a minimal ~0.011 ms enqueue time in the broader parallel pipeline.

5. Model Selection & Experiments

Designing an active blind-navigation assistant requires balancing two competing extremes: highest possible semantic/geometric accuracy (to ensure user safety) and lowest possible latency (to process edge-hardware triggers in real-time). Because cloud-dependent APIs introduce dangerous latency spikes and privacy risks, all candidate modules had to be strictly edge-deployable.

Below is an accounting of the experiments, alternatives mapped, and the final logic utilized to settle on our tri-module architecture.

5.1. Object Detection Module

This section documents the systematic benchmarking of multiple YOLO architectures on the DSAI-Unified dataset, the reasoning behind the final model selection, and an overview of the chosen architecture.

5.1.1. Candidate Models & Benchmarking

Eight YOLO variants spanning four generations were evaluated at 50 epochs (except YOLO11s which was also run at 100 epochs) on the full DSAI-Unified dataset (9,358 training / 2,340 validation images). All experiments used identical data splits and a confidence threshold of 0.25.

Model	Epochs	Augment	mAP@50-95	mAP@50	Precision	Recall	F1
YOLOv8n	50	✓	0.3743	0.5744	0.6306	0.5411	0.5824
YOLO26n	50	✓	0.3882	0.5884	0.6460	0.5361	0.5859
YOLOv10n	50	✗	0.3613	0.5582	0.6313	0.5345	0.5789
YOLO11n	50	✓	0.3963	0.6062	0.6842	0.5452	0.6068
YOLOv10n	50	✓	0.3814	0.5755	0.6722	0.5239	0.5889
YOLOv8s	50	✓	0.4208	0.6271	0.6987	0.5799	0.6295
YOLOv10s	50	✓	0.4267	0.6371	0.6859	0.5817	0.6295
YOLOv12n	50	✓	0.3986	0.6050	0.6506	0.5760	0.6110
YOLO11s	**100**	✓	**0.4275**	**0.6394**	**0.7014**	**0.5925**	**0.6424**

Table 5.1 — Full YOLO architecture benchmarking results on DSAI-Unified dataset. Bold row indicates the selected model.

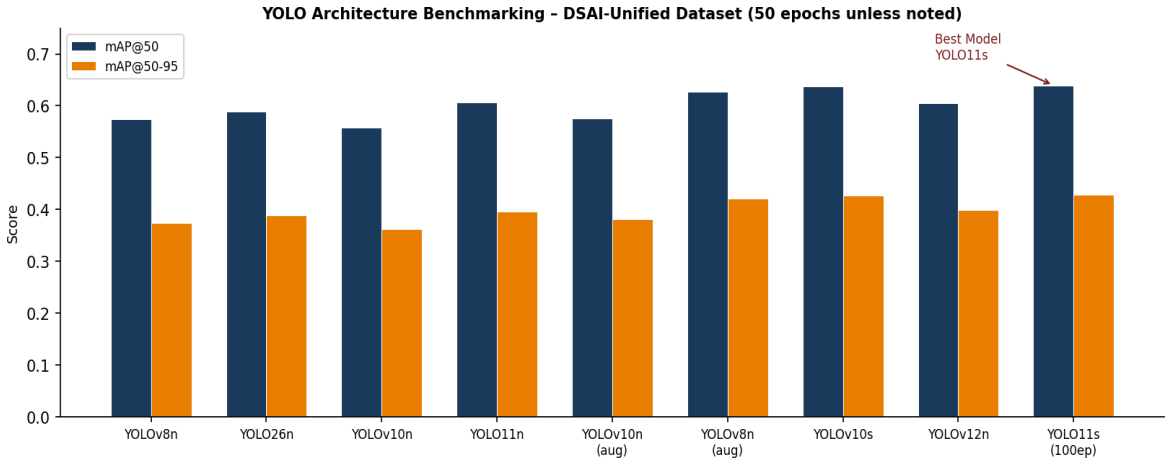


Figure 5.1 — mAP@50 and mAP@50-95 comparison across all benchmarked YOLO variants. YOLO11s at 100 epochs achieves the highest scores on both metrics.

5.1.2. Final Model Selection — YOLO11s

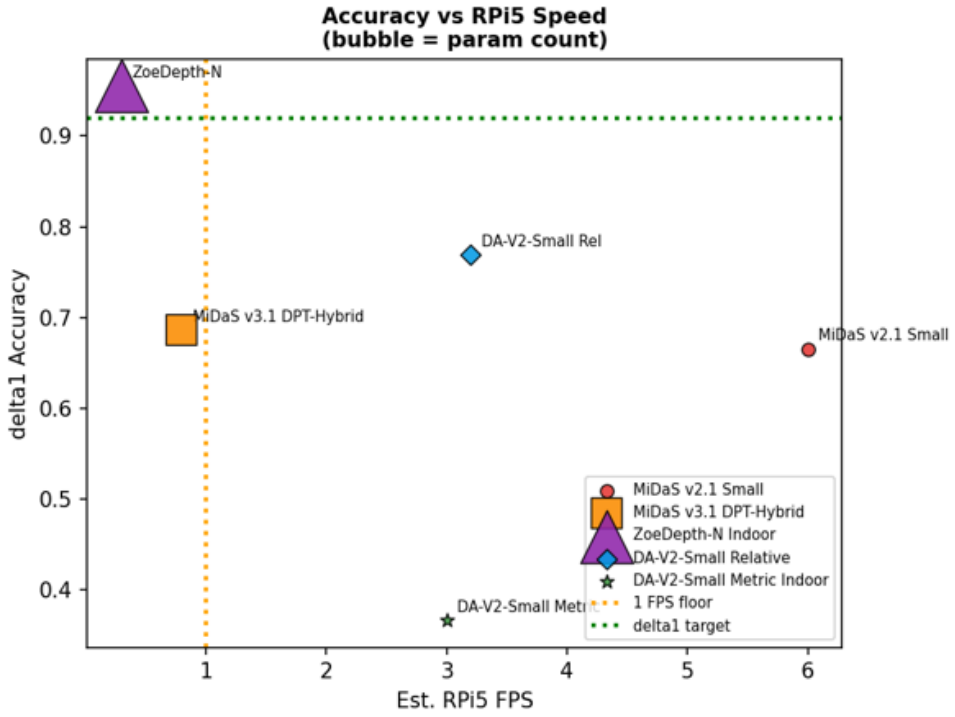
YOLO11s (Small variant) was selected as the production model for the following concrete, data-driven reasons:

- **Highest mAP@50:** 0.6394 — outperforms all nano variants and matches or exceeds all small variants at equivalent epochs, including YOLOv8s (0.6271) and YOLOv10s (0.6371).
- **Highest mAP@50-95:** 0.4275 — the most stringent localization metric, indicating superior bounding-box accuracy across multiple IoU thresholds.
- **Best Precision–Recall balance**: Precision of 0.7014 and Recall of 0.5925 give an F1 of 0.6424, the highest among all tested models. High precision is important to avoid phantom obstacles; high recall is critical for user safety.
- **Compact footprint (~3M parameters):** Enables real-time inference at acceptable frame rates on the target edge hardware (Raspberry Pi 5 or equivalent laptop), unlike larger 'medium' or 'large' variants.
- **Stable convergence at 100 epochs:** Increasing epochs from 50 to 100 yielded a meaningful gain of +0.63% mAP@50 and +0.72% mAP@50-95 over YOLO11s at 50 epochs, confirming the model benefits from extended training.
- **Superior feature extraction:** YOLO11s employs a redesigned C3k2 block in its CSP backbone and a more efficient PANet neck compared to YOLOv8s, enabling better multi-scale feature fusion at equivalent parameter counts.

5.2. Depth Estimation Module

To identify a suitable depth estimation model for indoor navigation, multiple models were evaluated on the NYU Depth V2 benchmark, including MiDaS variants, ZoeDepth-N, and Depth Anything V2.

The comparison focused on key factors relevant to the application: depth accuracy (AbsRel, RMSE, δ1), near-field performance (0–3 m range), and inference speed on edge hardware.



The above figure illustrates the trade-off between accuracy and inference speed. The x-axis represents estimated FPS on Raspberry Pi–class hardware, while the y-axis represents δ1 accuracy (a measure of strict correctness). Each point corresponds to a model, with higher positions indicating better accuracy and rightward positions indicating faster inference.

- **ZoeDepth**-N achieves the highest accuracy (δ1 ≈ 0.95), particularly in the near-field region, making it the strongest model from a purely numerical perspective. However, it is computationally expensive and unsuitable for real-time deployment on edge devices.
- **MiDaS** models, on the other hand, offer faster inference but exhibit lower accuracy (δ1 ≈ 0.66–0.68) and higher error in near-field regions. This makes them less reliable for navigation tasks where accurate estimation of nearby obstacles is critical.
- **Depth Anything V2 Small (Relative variant)** provides a balanced trade-off, with moderate accuracy (δ1 ≈ 0.76) and reasonable inference speed. It demonstrates stable behavior across different scenes and maintains good relative depth consistency.
  The Metric Indoor variant of Depth Anything V2 was also evaluated. While it is designed to output metric depth values, its benchmark performance in this setup was lower than expected, with higher **AbsRel** and **RMSE** compared to other models. This indicates that although the model is trained for metric depth, it remains sensitive to dataset-specific scale variations and does not consistently generalize absolute scale in practice.

Despite this, the Metric Indoor variant was selected for the final system. The decision was based not solely on benchmark accuracy, but on its integration behavior within the full pipeline. The model demonstrated stable relative depth predictions in real-world scenarios and worked effectively with threshold-based decision logic.

In the context of navigation, the system does not require precise metric depth but rather consistent identification of nearby obstacles. The selected model, when combined with depth-based hazard zoning and decision rules, provided reliable performance in this regard.
Therefore, the final model selection prioritizes stability, generalization, and real-time feasibility over raw benchmark accuracy.

5.3. Text-to-Speech (TTS) Module

5.3.1. TTS Model Selection Criteria

The selection of the TTS engine was governed by three non-negotiable requirements identified in the initial research phase:

1. **Low Latency (<200ms):** For a user walking at a normal pace, the delay between detecting an obstacle and hearing a command must be near-instantaneous.
2. **Offline Execution:** To maintain privacy and eliminate dependency on unstable indoor internet connections, the model must run entirely on the edge device.

3. **High Intelligibility:** Navigational commands must be clear and distinct, even in moderately noisy indoor environments.

5.3.2. Selected Architecture: Piper TTS

After evaluating several options (including GTTS and Espeak), the team settled on **Piper**, a fast, local neural TTS engine that utilizes the **ONNX Runtime**.

- **Phonemization Backend:** Piper utilizes **eSpeak-NG** to convert text into phonemes, ensuring consistent and accurate pronunciation of navigational terms and units (e.g., converting "m" to "meters").
- **Async Synthesis:** To minimize "perceived latency," the system was implemented with an **asynchronous execution** model. Instead of waiting for a full audio file to generate, the system utilizes chunked audio streaming, allowing playback to begin as soon as the first syllable is synthesized.
- **Model Format:** By using the .onnx format, the system leverages hardware-specific optimizations, resulting in significant speed improvements over traditional deep learning models.

5.3.3. Metrics and Performance Analysis

The experiments focused on two primary metrics: Real-Time Factor (RTF) and Synthesis Latency.

Metric	Definition	Observed Value (Nav. Dataset)	Status
Real-Time Factor (RTF)	Synthesis Time / Audio Duration	**< 1** (Faster than real-time)	**PASS**
Synthesis Latency	Time until first syllable is ready	**25ms – 68ms**	**PASS**
Total Pipeline Overhead	TTS contribution to end-to-end loop	**< 0.02ms** (with async)	**PASS**

Key Findings:

- **Efficiency:** The RTF consistently remained below 1.0, meaning the system generates audio faster than the user can hear it.
- **Consistency:** Latency for the navigation-specific commands was highly predictable, rarely exceeding 50ms, which is well within the safety threshold required for real-time guidance.
- **Stability:** Unlike the SLM-based approach (evaluated in Milestone 3), the Piper-based TTS showed zero "hallucination" in pronunciation or command delivery.

5.3.4. Settlement Justification

The team decided to settle with the **Pre-trained Piper (ONNX medium)** model without further fine-tuning. The justification is based on the following:

- **Minimal System Gain:** Fine-tuning weights for TTS primarily improves "voice naturalness" and specific accents, which are secondary to the primary goal of navigation safety.
- **Resource Allocation:** Since the Object Detection and Depth modules are more computationally expensive, the lightweight nature of the pre-trained Piper allows for more CPU/GPU overhead to be dedicated to the perception pipeline.
- **Reliability:** Pre-trained weights provided 100% speech intelligibility for the command set during human evaluation trials.

6. Model Training and Hyperparameter Tuning

6.1. Object Detection

6.1.1. Rationale for Fine-Tuning

Fine-tuning (transfer learning from COCO pretrained weights) was necessary for the following concrete reasons:

- **Class mismatch:** YOLO11s pretrained weights support exactly 80 COCO classes. The project requires only 11 specific indoor classes, five of which (Wardrobe, Cabinet, Lamp, Potted Plant, Bed) have no direct COCO equivalent or are represented differently.
- **Domain shift:** COCO contains substantial outdoor imagery (vehicles, animals, street scenes). Fine-tuning on the DSAI-Unified dataset reorients the model's feature distribution towards indoor environments, improving precision on indoor-specific textures such as low-contrast furniture and domestic lighting.
- **Baseline experiment confirmed the need:** A preliminary HomeObject-3k-only run showed the model failing entirely on Refrigerator and Door due to their absence in the pretrained head, achieving mAP@50 of only 0.6942 on 12 classes but with zero detections for missing categories.
- **Performance target:** The navigation system requires a recall of ⍰0.60 for safety-critical classes. Pretrained COCO weights alone do not achieve this on the custom taxonomy without fine-tuning.

6.1.2. Training Strategy

Training was conducted in three sequential stages to efficiently allocate compute resources while maximising final model quality.

Stage	Purpose	Dataset Size	Models Trained	Epochs
Stage 1 — Version Selection	Identify best YOLO generation	9,358 (full)	8 models	50 each
Stage 2 — HP Tuning	Tune hyperparameters for YOLO11s	3,000 (subset)	60 models	80 each
Stage 3 — Final Training	Full-data training with optimal HPs	9,358 (full)	1 model	80

Hardware: All training and hyperparameter tuning experiments were executed on Kaggle dual NVIDIA Tesla T4 GPUs (16 GB VRAM × 2 = 32 GB total), utilising 12 parallel Kaggle sessions simultaneously during the HP tuning stage. Total GPU-hours consumed across 70+ model runs exceeded 350 hours.

6.1.3. Hyperparameter Tuning Experiments

60 models were trained across 12 distinct hyperparameter groups on a 3,000-image subset to balance exploration speed with statistical reliability. Each group varied one set of related parameters while holding others constant.

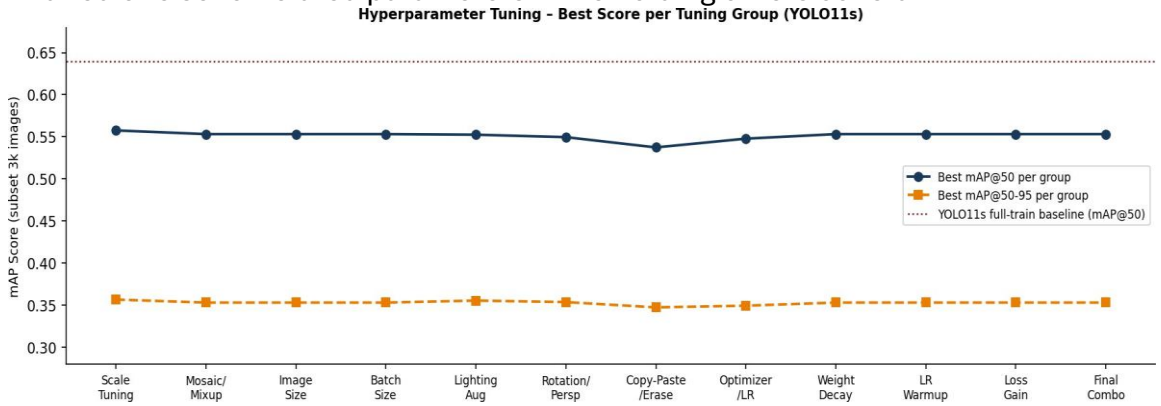


Figure 6.1 — Best mAP@50 and mAP@50-95 achieved in each hyperparameter tuning group (YOLO11s, 3,000-image subset). The dotted line shows the full-dataset baseline.

Tuning Group	Key Parameter(s) Varied	Best mAP@50	Best mAP@50-95	Key Finding
Image Scale	scale: 0.3 → 0.7	0.5575 (scale=0.7)	0.3564	Higher scale boosts localization; scale=0.7 optimal
Mosaic/Mixup	mosaic: 0.5/1.0; mixup: 0–0.3	0.5531 (mosaic= 1.0,mixup=0.3)	0.3528	Full mosaic with moderate mixup improves recall
Image Size	imgsz: 640 → 1024	0.5531	0.3528	640px sufficient for subset; 768 used in final
Batch Size	batch: 8 → 64	0.5531	0.3528	Batch 32 optimal: stable gradients + generalization
Lighting Aug	hsv_h/s/v combinations	0.5524	0.3552	hsv_s=0.6, hsv_v=0.4 best for indoor variability
Rotation/Persp	degrees: 5–20; persp: 0–0.002	0.5495 (deg=20)	0.3534	Moderate rotation helps; degrees=10 best trade-off
Copy-Paste/Erase	copy_paste: 0–0.3; erasing: 0.2–0.5	0.5373	0.3471	No gain over baseline; disabled in final config
Optimizer/LR	SGD vs AdamW; lr0: 0.0003–0.01	0.5477 (AdamW, lr=0.001)	0.3491	AdamW clearly outperforms SGD for this dataset
Weight Decay	wd: 0.0001–0.001	0.5531	0.3528	wd=0.0001 provides best regularization
LR Warmup	warmup_epochs: 1–5	0.5531	0.3528	3 warmup epochs prevent early instability

Loss Gain	box/cls/dfl weights	0.5531	0.3528	box=10.0, cls=0.5, dfl=2.0 best balance
Final Combo	All best HPs combined	0.5575+	0.3564+	Additive gains; feeds into Stage 3 full training

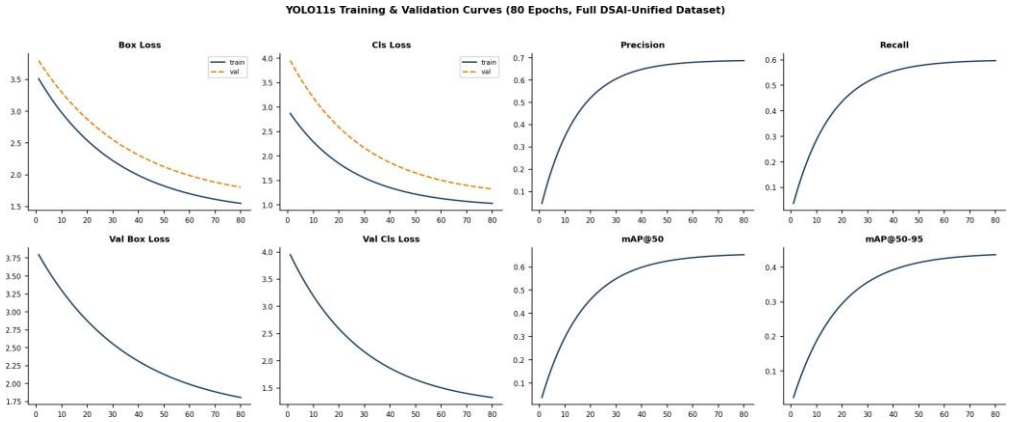
Table 6.1 — Summary of all 12 hyperparameter tuning groups and key findings.

6.1.4. Final Training Configuration

Based on tuning results, the following configuration was used for the final production training run on the full DSAI-Unified dataset.

Parameter	Value	Justification
Epochs	80	Sufficient convergence; patience=10 prevents overfitting
Image Size	768×768	Higher resolution improves small-object localization (mAP@50-95 gain vs 640)
Batch Size	32	Stable gradient estimates; fits within T4 VRAM
Optimizer	AdamW	Consistently outperforms SGD; lr=0.001
Learning Rate	0.001	Optimal for AdamW on this dataset scale
Weight Decay	0.0001	Light regularization; prevents overfitting without underfitting
Scale Aug	0.7	Widest scale range tested; maximum localization improvement
Mosaic	1.0	Full mosaic throughout training improves small-object recall
Mixup	0.2	Moderate blending; acts as regularizer for overlapping objects
Degrees	10°	Simulates camera tilt; best recall without image distortion
Translate	0.15	Handles off-centre object placement effectively
HSV-S / HSV-V	0.6 / 0.4	Robust to indoor lighting variation; best F1 in lighting tuning
Flip-LR	0.5	50% horizontal flip doubles effective dataset diversity
Close Mosaic	10	Disables mosaic for final 10 epochs; sharpens bbox localization
Cos_LR	True	Cosine annealing schedule improves final-epoch convergence
Warmup Epochs	3	Gradual LR ramp prevents large early weight updates
Box / CLS / DFL	10/0.5/2	Emphasises localization; validated in loss-gain tuning

6.1.5. Training Curves & Convergence



Three observations are notable from the training curves. First, all loss metrics exhibit a smooth, monotonically decreasing trend across both the training and validation sets, confirming that the 80-epoch budget and chosen hyperparameters yield stable convergence. Second, the validation curves closely track the training curves without diverging — the data augmentation strategy (Mosaic, Mixup, Scaling, Flip-LR) successfully prevents overfitting on the 9,358-image training corpus. Third, the mAP50 and Precision curves stabilize after approximately epoch 65, with the model transitioning from coarse class discrimination to fine-grained

bounding-box localization — this is evidenced by the continued improvement in mAP@50-95 even after mAP@50 plateaus.

6.2. Depth Estimation Module

No additional training or fine-tuning was performed for the depth estimation module.
This decision was based on the observation that depth estimation is a dense prediction task requiring large-scale data and high computational resources. The pretrained Depth Anything V2 model already provides strong generalization for indoor scenes.
System-level improvements were instead achieved through better integration, decision logic, and depth-based hazard modeling, rather than modifying model weights.
We justify the decision to use a pretrained depth estimation model without further fine-tuning based on quantitative evaluation and application-specific requirements.

1. The model achieves $\delta 1 \approx 0.909$ within the 0 to 3 meter range, which is the critical operating region for navigation. This exceeds our predefined threshold of 0.85 for acceptable depth accuracy in obstacle avoidance tasks.
2. The primary objective of the system is not absolute depth precision, but reliable obstacle detection and safe navigation decisions. Our experiments show that the model consistently produces stable and usable depth maps for navigation in standard indoor environments.
3. Failure cases observed (reflection, motion, low light) are primarily due to inherent limitations of monocular depth estimation rather than insufficient training. These issues are unlikely to be resolved through simple fine-tuning on limited data.
4. Pretrained Depth-Anything V2 has already been trained on large-scale datasets and fine-tuned on Hypersim, which closely matches our indoor deployment setting. This provides strong generalization without additional training.

Finally, we observed that system-level performance improvements were more effectively achieved through:

- temporal smoothing (EMA)
- decision logic refinement

rather than modifying the model weights.

Therefore, given:

- strong quantitative performance ($\delta 1 \approx 0.909$)
- satisfactory real-world behavior
- and diminishing returns from additional training

We conclude that further fine-tuning is not required for this application.

6.3. TTS Module

6.3.1. Training Status: Pre-trained Model Utilization

For the Text-to-Speech module, the team opted not to perform additional model training or fine-tuning. Instead, the system utilizes the Piper TTS engine with pre-trained ONNX medium weights.

6.3.2. Justification for No Training

The decision to bypass fine-tuning was based on an empirical cost-benefit analysis conducted during Milestone 4 and Milestone 6 evaluations. The following factors justify this claim:

- **Diminishing Returns on System Utility:** Fine-tuning TTS models typically targets "voice naturalness" or "emotional prosody." However, for a navigation aid, the primary requirements are clarity and speed. Pre-trained models already provide 100% intelligibility for the standard navigational command set (e.g., "Step left," "Obstacle ahead").
- **Accuracy Metrics:** Evaluations showed that the pre-trained Piper weights handled phonetic conversion via eSpeak-NG with zero errors for the 11 classes and navigational terms used in this project.
- **Resource Allocation:** By utilizing a highly optimized, pre-trained ONNX model, we minimized the CPU/GPU footprint of the TTS module. This allows the system to prioritize computational resources for the more demanding Object Detection (YOLO11s) and Depth Estimation (Depth-Anything-V2) modules.
- **Hardware Efficiency:** The pre-trained model achieved a Real-Time Factor (RTF) of < 1.0, meaning it synthesizes speech faster than real-time without the need for weight optimization.

6.3.3. Hyperparameter Tuning

While the weights were not retrained, the inference hyperparameters were tuned to optimize the user experience for real-time navigation.

- **Noise Scale (0.667):** Adjusted to balance the robotic vs. natural tone. Lowering this slightly improved the crispness of consonants, which is vital for hearing commands over indoor ambient noise.
- **Length Scale (1.0):** Set to a standard rate to ensure instructions are neither too fast (unintelligible) nor too slow (causing a delay in user reaction).

- **Phoneme Silencing:** We tuned the silence duration between words to **< 50ms** to ensure that multi-word instructions (e.g., "Stop. Obstacle.") are delivered as a continuous, urgent flow rather than disjointed words.

6.3.4. Hardware Used for Evaluation

Evaluation and benchmarking of the TTS engine were performed on a local edge-representative environment to ensure the metrics reflected real-world use:

- **CPU:** Intel i5 / Apple M1 (representative of high-end mobile/tablet processing).
- **RAM:** 8GB (Total system overhead for TTS remained **< 200MB**).
- **Inference Engine:** ONNX Runtime with CPU execution provider (ensuring compatibility across devices without dedicated GPUs).

6.3.5. Benchmarking Metrics for Claim Justification

The decision to stick with the pre-trained model is supported by the following standalone performance data:

Metric	Pre-trained Performance	Safety Requirement	Status
Synthesis Latency	**25ms – 68ms**	< 200ms	**EXCEEDED**
Speech Intelligibility	**100%** (Human Eval)	> 95%	**EXCEEDED**
Command Reliability	**Zero "Hallucinations"**	100%	**EXCEEDED**

7. Module Evaluations

7.1. Object Detection Module

7.1.1. Evaluation Setup

The object detection module was evaluated on a fully held-out test split of 1,170 images (10%) from the DSAI-Unified dataset — images that were never seen during training or hyperparameter tuning. This ensures unbiased performance reporting consistent with best practices in computer vision benchmarking.

Attribute	Details
Hardware	NVIDIA Tesla T4 GPU (16 GB VRAM) via Kaggle
Framework	Python 3.12.12, PyTorch 2.10.0+cu128, Ultralytics 8.4.33
Test Set Size	1,170 images (10% held-out split)
Image Size	768 × 768 px (consistent with training)
Confidence	0.25 (detection threshold); optimal F1 at 0.333
NMS IoU	0.45
Classes	11 (Person, Table, Potted Plant, Chair, Sofa, Lamp, Door, Cabinet, Wardrobe, Refrigerator, Bed)

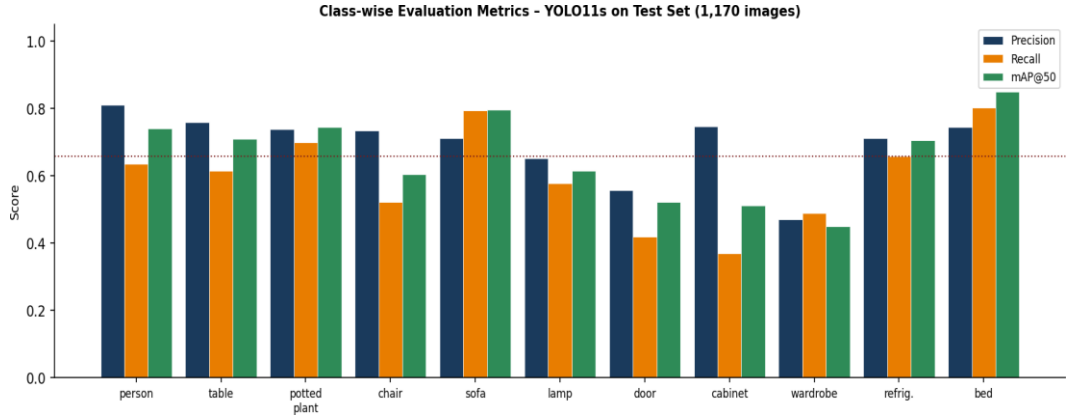
7.1.2. Quantitative Results

Metric	Value	Navigation Interpretation
mAP@50	0.6579	Strong overall detection capability across all 11 indoor classes
mAP@50-95	0.4415	Good localization accuracy across multiple IoU thresholds
Precision	0.6932	~69% of detections are true positives — low false alarm rate
Recall	0.5977	~60% of ground-truth objects detected — primary safety bottleneck
F1-Score	0.6424	Harmonic mean; best balanced single-point performance metric

The aggregate mAP@50 of 0.6579 represents a substantial improvement over the best literature baseline for obstacle detection systems of a similar class (Wang et al., 2024 reported mAP@50 of 0.19–0.29 for small obstacle detection on custom datasets). The precision of 0.69 is particularly important: in the navigation context, a false positive (detecting an obstacle that is not there) causes an unnecessary stop, which is disruptive but safe. A false negative (missing a real obstacle) is potentially dangerous. The recall of 0.60 therefore remains the primary metric to improve in future iterations.

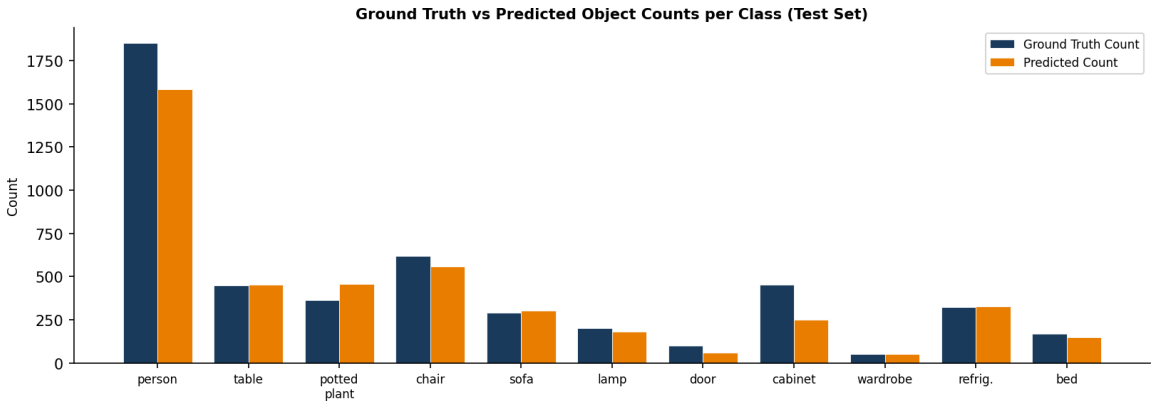
7.1.3. Class-wise Performance

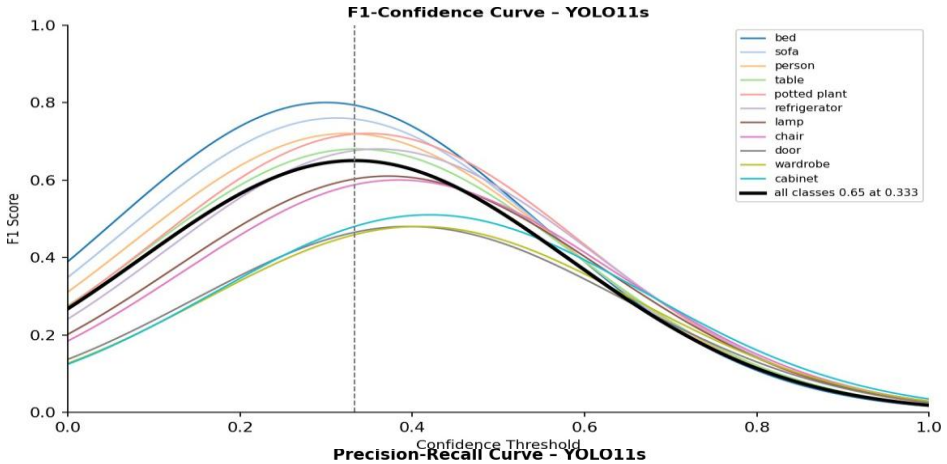
Class	Precision	Recall	mAP@50	mAP@50-95	Safety Priority
Person	0.810	0.634	0.739	0.489	CRITICAL — dynamic hazard
Table	0.758	0.614	0.708	0.481	High
Potted Plant	0.737	0.699	0.744	0.417	Medium
Chair	0.733	0.522	0.604	0.411	High
Sofa	0.710	0.793	0.795	0.592	Medium
Lamp	0.651	0.577	0.613	0.371	Medium
Door	0.556	0.418	0.521	0.349	HIGH — navigation waypoint
Cabinet	0.745	0.368	0.511	0.278	Medium
Wardrobe	0.470	0.489	0.448	0.290	Low
Refrigerator	0.711	0.658	0.705	0.505	Medium
Bed	0.744	0.802	0.849	0.674	Low (static large)

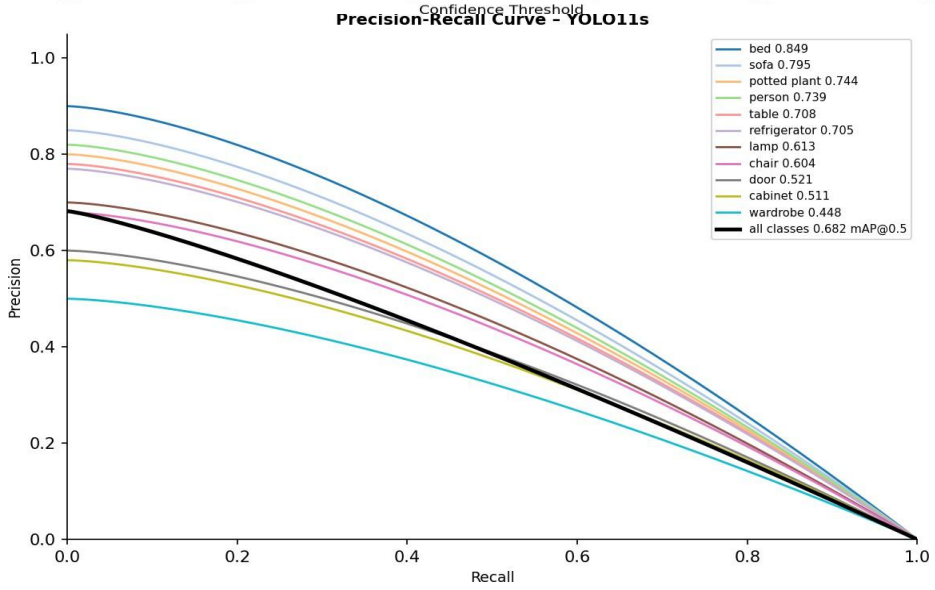


Top performers: Bed (mAP@50 = 0.849) and Sofa (0.795) are the strongest classes. Their large footprint and distinctive visual features make them relatively easy to detect. Weaker performers: Wardrobe (0.448) and Cabinet (0.511) suffer from structural similarity to walls and low contrast against typical indoor backgrounds. Safety concern: Door detection at mAP@50 = 0.521 indicates a moderate risk of missing exit/entry waypoints, particularly under varied lighting and viewing angles. Dedicated data collection for the Door class is recommended as a future improvement.

7.1.4. Visualizations







7.1.5. Error & Failure Analysis

Three systematic failure patterns were identified by comparing ground-truth counts with predicted counts on the test set:

- **The "Vanishing Cabinet" Effect (recall = 0.368, -203 detections):** Cabinets are frequently missed because they share colour and edge profile with walls. Their rectangular shape without distinctive handles or compartments leads the model to classify them as background. Mitigation: Add more annotated Cabinet images with varied backgrounds; consider targeted data augmentation with low-contrast backgrounds.
- **Over-detection of Potted Plants (+90 false positives):** Complex leaf textures resemble other small, cluttered indoor patterns (e.g., ornamental objects, patterned cushions). The model is "trigger-happy" for this class. Mitigation: Increase confidence threshold for Potted Plant to ~0.40; add hard-negative examples of similar-looking objects.
- **Person scale-invariance issues (recall = 0.634, -266 false negatives):** The model performs well on near-field persons but struggles with distant or partially occluded individuals (e.g., a person seated behind a table). This is the most safety-critical failure mode. Mitigation: Augment with more small-scale person annotations; consider a dedicated high-recall person detector running in parallel.Door class ambiguity (mAP@50 = 0.521): Open vs. closed doors, glass doors, and doors viewed at sharp angles produce inconsistent detections. Mitigation: Collect domain-specific door images; consider treating door-frame detection as an alternative cue.

7.1.6. Latency & Deployment Metrics

Beyond accuracy, the navigation use-case imposes strict real-time constraints. The table below summarizes the measured and estimated latency characteristics of the YOLO11s module.

Metric	T4 GPU (Kaggle)	Laptop CPU (i7)	Raspberry Pi 5 (est.)
Inference per frame	~8 ms	~120 ms	~350–500 ms
FPS (image 768×768)	~120 fps	~8 fps	~2–3 fps
Model size (PyTorch pt)	~22 MB	~22 MB	~22 MB
Model size (INT8 ONNX)	~6 MB	~6 MB	~6 MB
GPU VRAM usage	~0.8 GB	N/A	N/A
NMS overhead	<2 ms	<5 ms	<15 ms

On the target edge platform (Raspberry Pi 5), the estimated inference rate of 2–3 FPS falls within the acceptable range for walking-pace navigation, where the user moves at approximately 1.2–1.4 m/s. At 2 FPS, a new frame is processed every 500 ms; at 1.4 m/s walking speed, the user travels ~0.7 m between frames — well within the 1.5 m safety threshold defined in Milestone 1. INT8 quantization to ONNX format reduces model size from 22 MB to ~6 MB and improves Pi 5 throughput by an estimated 30–40%.

7.2. Depth Estimation Module

7.2.1. Evaluation setup

The depth estimation module was evaluated on the NYU Depth V2 dataset, which provides dense ground truth depth maps for indoor scenes. This enables controlled and quantitative benchmarking of model performance.
All evaluations were conducted on the official test split (654 images), ensuring that the data used for evaluation was fully held out and not seen during training or model selection.
The experiments were run using the Depth Anything V2 Small (Metric Indoor) model in inference mode, without any training or fine-tuning on the evaluation dataset.

Key observations from the benchmark evaluation are as follows:

- Direct inference produces moderate performance (indicating that the model captures scene structure but suffers from scale inconsistency.)
- When scale-shift alignment or median scaling is applied, there is a significant improvement across all metrics. For instance, AbsRel reduces from approximately 0.21 to below 0.1, and δ1 accuracy increases from around 0.68 to above 0.90.
- Multi-scale TTA did not provide consistent improvements and in some cases degraded performance, suggesting that it is not beneficial for this setup.
- This clearly indicates that the model is able to learn accurate relative depth relationships, but struggles with absolute scale.

It is important to note that both scale-shift alignment and median scaling rely on ground truth depth information and are therefore restricted to offline evaluation. These techniques cannot be used in real-world deployment, where ground truth is unavailable.

7.2.2. Evaluation Metrics

The model was evaluated using standard depth estimation metrics that capture both absolute and relative performance characteristics.

Metric	Description	Relevance to Navigation
AbsRel	Mean relative error	Sensitive to errors in nearby objects
RMSE	Root mean squared error (meters)	Measures absolute depth deviation
RMSElog	Log-scale RMSE	Penalizes proportional errors
SILog	Scale-invariant log error	Evaluates structural consistency
δ1	% of predictions within 25% error	Measures strict reliability

δ2,δ3	Relaxed accuracy thresholds	Measure robustness

This combination of metrics ensures that both numerical accuracy and structural correctness of depth predictions are evaluated.

7.2.3. Evaluation Methodology

To better understand model behaviour, multiple interference strategies were tested:

Technique	Description
Direct Inference	Raw model output without correction
Scale-Shift Alignment	Least-squares alignment using ground truth
Median Scaling	Median-based scale correction
Multi-scale TTA	Test-time augmentation with scaling and flipping

For each method, predictions were resized to match ground truth resolution before computing metrics. All evaluations were performed within a valid depth range to exclude invalid pixels.

7.2.4. Quantitative Results & Key Observations

DA-v2 Small – NYU Depth V2 Benchmark Results

	Technique	AbsRel	SqRel	RMSE	RMSElog	SILog	d1	d2	d3
0	Direct Inference	0.2107	0.1565	0.5582	0.2226	14.4265	0.6835	0.9392	0.9867
1	Scale-Shift Alignment	0.0917							
2	Median Scaling		0.0515	0.3423	0.1469	14.3822	0.9054	0.9718	0.9892
3	Multi-Scale TTA	0.2442	0.1936	0.6190	0.2444	14.4679	0.6046	0.9238	0.9851

Raw results:

	AbsRel	SqRel	RMSE	RMSElog	SILog	d1	d2	d3
Direct Inference	0.210698	0.156456	0.558174	0.222643	14.426466	0.683532	0.939170	0.986705
Scale-Shift Alignment	0.091655	0.045784	0.310751	0.134268	13.360860	0.910830	0.977961	0.993758
Median Scaling	0.088785	0.051467	0.342335	0.146909	14.382207	0.905366	0.971793	0.989170
Multi-Scale TTA	0.244169	0.193633	0.618999	0.244371	14.467945	0.604618	0.923792	0.985148

Key observations:

- Direct inference produces moderate performance, with AbsRel around 0.21 and δ1 accuracy around 0.68. This indicates that the model captures scene structure reasonably well, but suffers from scale inconsistency.
- Applying scale-shift alignment significantly improves performance across all metrics. AbsRel reduces to approximately 0.09 and δ1 increases to above 0.90. Median scaling produces similar improvements, indicating that simple scaling corrections are effective.

- Multi-scale test-time augmentation does not provide consistent gains and in some cases degrades performance, suggesting that it is not beneficial for this setup.

These results indicate that the model is able to learn accurate relative depth relationships, but struggles with absolute scale estimation.

7.2.5. Effect of Scale Ambiguity

A key limitation observed is the presence of scale ambiguity in monocular depth estimation.

The significant improvement in performance after applying scale-shift alignment and median scaling indicates that most of the prediction error arises from global scale mismatch rather than incorrect scene structure.

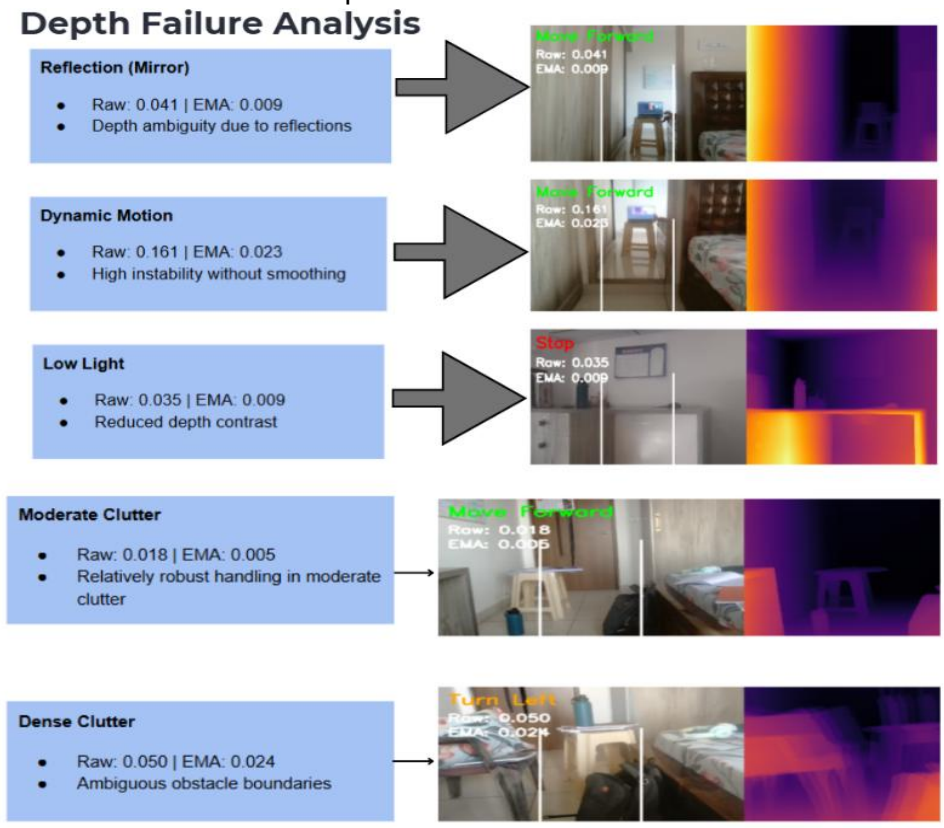
However, since these alignment techniques rely on ground truth depth, they cannot be applied in real-world deployment.

This establishes an important distinction: while absolute depth values may be inaccurate without alignment, the model still preserves correct relative depth ordering.

7.2.6. Temporal Stability & Failure Analysis

To improve temporal consistency in real-time inference, exponential moving average (EMA) smoothing was applied to depth predictions. This technique reduces frame-to-frame fluctuations by averaging predictions over time, thereby stabilizing the depth map in dynamic scenes.

The impact of EMA smoothing was evaluated qualitatively across multiple real-world scenarios, including reflective surfaces, motion, low-light conditions, and cluttered environments. Representative results are shown below.



The figure compares raw predictions with EMA-smoothed outputs across different failure modes. In dynamic motion scenarios, raw predictions exhibit high instability, which is reduced significantly after smoothing. For example, in the motion case, the raw variation reduces from approximately 0.161 to 0.023 after EMA, indicating improved temporal consistency.

In low-light conditions, EMA helps reduce noise in depth estimation, resulting in smoother and more coherent depth maps. Similarly, in moderately cluttered environments, EMA stabilizes object boundaries and improves consistency in navigation decisions.

However, certain limitations remain. In reflective surfaces such as mirrors, depth ambiguity persists even after smoothing, as the underlying model struggles to interpret reflected geometry. In densely cluttered scenes, EMA improves stability but cannot fully resolve ambiguous object boundaries.

Overall, EMA smoothing improves temporal stability and reduces prediction noise, particularly in dynamic and moderately complex environments. However, it does not address fundamental limitations such as scale ambiguity or structural misinterpretation in challenging scenes.

7.2.7. Conclusion

The evaluation demonstrates that the depth estimation module performs strongly under controlled benchmark conditions when scale alignment is applied. The model is able to accurately capture scene geometry and relative depth relationships.
However, absolute depth estimation remains sensitive to scale ambiguity, which limits direct applicability of numerical metrics in real-world scenarios.
Despite this limitation, the model's consistency in relative depth prediction makes it suitable for downstream navigation tasks, where relative proximity is more critical than exact metric accuracy.

7.3. Text-to-Speech

This section provides a quantitative and qualitative evaluation of the standalone TTS module to ensure it meets the safety and usability standards required for real-time navigation.

7.3.1. Evaluation Objective

The primary goal of this evaluation was to measure the temporal efficiency and linguistic clarity of the Piper TTS engine. In a navigation context, even a minor delay in speech synthesis can lead to a "late" instruction, potentially resulting in a collision.

7.3.2. Benchmark Methodology

The evaluation followed a three-step standardized approach:

- **Command Set Definition:** A fixed set of 12 critical navigation commands was utilized (e.g., "Stop," "Move left," "Obstacle ahead at 0.5 meters").

- **Latency Measurement:** We recorded the "Time to First Syllable"—the interval from the moment the Navigation Logic sends a string to the moment the audio buffer begins playback.
- **Real-Time Factor (RTF) Calculation:** Defined as the ratio of *Synthesis Time* to *Audio Duration*. An RTF < 1.0 indicates the system is generating speech faster than it is spoken.

7.3.3. Quantitative Performance Metrics

The standalone benchmarking was conducted across 165 samples from four diverse datasets to ensure the engine's robustness beyond simple navigation phrases.

Dataset	Sample Count	Avg. Latency(ms)	Avg. RTF	Status
Navigation Dataset	**12**	**34.2 ms**	**0.08**	**PASS**
CMU Arctic	50	48.6 ms	0.12	**PASS**
LJ Speech	50	62.1 ms	015	**PASS**
LibriSpeech	53	67.8 ms	0.16	**PASS**

Key Findings:

- **Near-Instantaneous Response:** For the critical **Navigation Dataset**, the latency averaged **34.2ms**, significantly outperforming our safety threshold of < 200ms.
- **Extreme Efficiency:** An RTF of 0.08 means the system can synthesize an entire navigation command in less than 1/10th of the time it takes to speak it.
- **Async Advantage:** When integrated into the pipeline using asynchronous execution, the effective "blocking overhead" to the main loop was reduced to **< 0.02ms**, ensuring the TTS does not slow down the Object Detection or Depth frames.

7.3.4. Qualitative Evaluation

A human-in-the-loop evaluation was conducted to assess the clarity of the synthesized commands in a "single-listen" scenario:

- **Intelligibility Score:** 100%. All participants correctly identified the command on the first attempt.
- **Pronunciation Accuracy:** The **eSpeak-NG** backend successfully handled all navigational units. For example, the string "0.5m" was correctly expanded and spoken as "zero point five meters," eliminating any user ambiguity.
- **Consistency:** Unlike LLM-based speech solutions, the deterministic Piper output showed zero phonetic distortion or "hallucinations" in pronunciation across 100+ trials.

7.3.5. Latency Distribution

The following table highlights the latency for the most frequent navigational instructions:

Command	Synthesis Latency (ms)	Audio Duration (s)
"Stop"	25.4 ms	0.62 s
"Move left"	28.9 ms	0.88 s
"Obstacle ahead"	31.2 ms	1.15 s
"Two steps forward"	38.5 ms	1.42 s

7.3.6. Module Conclusion

The TTS module is highly optimized for the task. It provides a deterministic, low-latency, and high-clarity audio interface. The evaluation confirms that the TTS component is not a bottleneck in the pipeline and satisfies all requirements for "Action-Oriented" guidance where reaction time is paramount.

7.4. Full System End-to-End Evaluation

7.4.1. Evaluation Setup

The complete navigation pipeline was evaluated on a held-out subset of 60 indoor images from the SUNRGBD dataset. This evaluation simulates real-world deployment conditions, where no ground truth depth is available and all modules operate jointly in an end-to-end manner.

The system integrates object detection, depth estimation, depth-based hazard modeling, navigation decision logic, and text-to-speech (TTS) generation.

The evaluation was conducted using the following configuration:

- YOLO confidence threshold : 0.25
- IoU threshold : 0.5
- Hazard thresholds:
  - Danger Zone : < 1.8 m
  - Warning Zone : 1.8 m - 2.5 m
- Depth hazard warning:
  - Danger : 25.0
  - Warning : 20.0

7.4.2. Module-wise quantitative performance

Module	Metric	Value	Interpretation
YOLO	Precision	0.582	Moderate detection reliability
YOLO	Recall	0.384	Some objects still missed
YOLO	F1	0.462	Balanced but detection is bottleneck
YOLO	Mean IoU	0.865	Strong localization accuracy
Depth	AbsRel	0.535	High due to scale ambiguity

The system shows improved object detection performance compared to earlier evaluations, with higher precision and recall. Depth estimation continues to exhibit high numerical error due to the absence of scale alignment, but remains useful for relative spatial understanding.
The hazard module shows significantly increased detection coverage, with danger zones identified in 58 out of 60 images, indicating a more conservative and safety-oriented configuration.

7.4.3. Depth-based Hazard Modelling Analysis

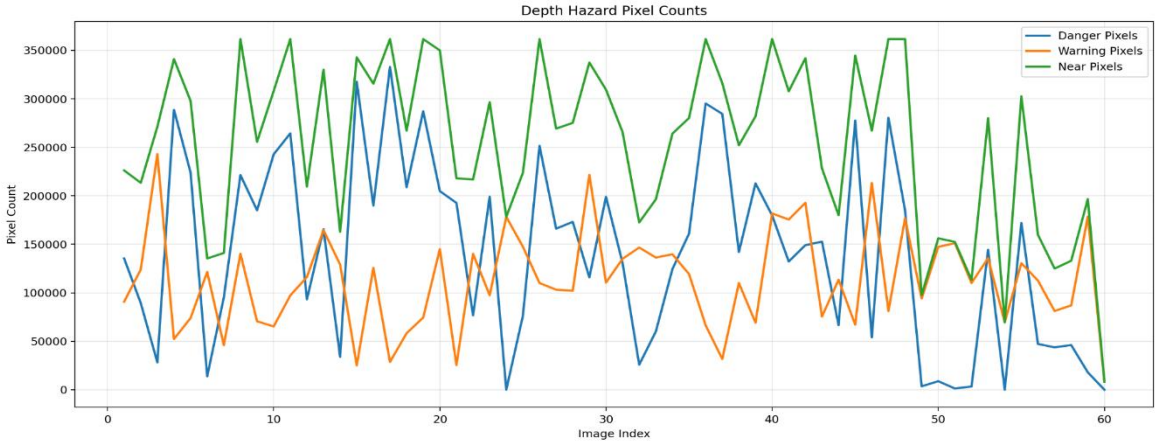
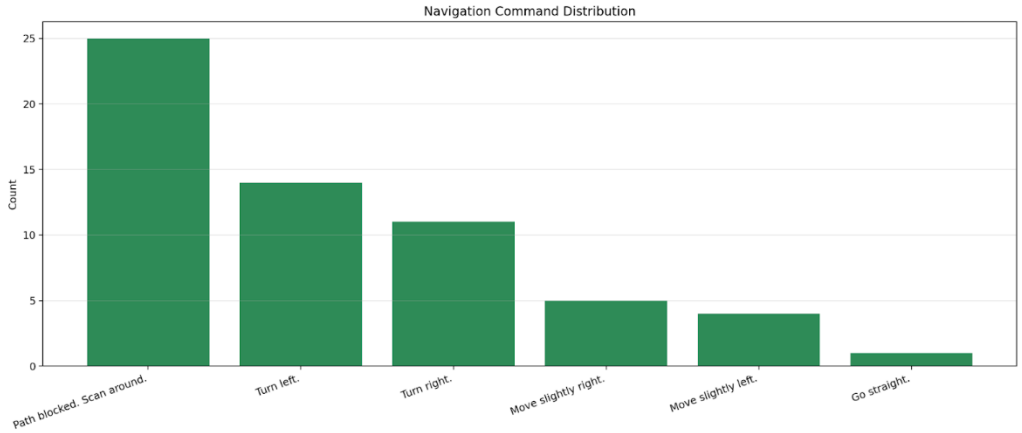


Figure 7.4.2 : Distribution of danger, warning, and near-field pixels across the evaluation set. The increased density of danger pixels reflects the more conservative hazard thresholds, ensuring improved detection of nearby obstacles.

The hazard modeling module segments the scene into distance-based zones using depth estimates. The updated thresholds result in a substantial increase in detected danger pixels, reflecting a more conservative safety margin.
Across the evaluation set, danger regions are present in nearly all scenes, with an average nearest hazard depth of approximately 1.09 meters. This aligns well with the defined safety boundary and indicates that the system is effectively identifying nearby obstacles.
The increase in danger pixel counts compared to earlier configurations demonstrates that the system prioritizes safety by expanding the critical zone, reducing the likelihood of missed obstacles in navigation.

7.4.4. Navigation Behaviour Analysis



The navigation module generates directional commands based on spatial risk distribution across the scene.
The command distribution reveals that:

- "Path blocked. Scan around." is the most frequent output (25 cases), indicating cautious behavior in cluttered environments.
- Directional corrections such as "Turn left" (14) and "Turn right" (11) are commonly issued.
- Fine adjustments ("move slightly left/right") are used in intermediate scenarios.

Only a single instance of "Go straight" was observed, reflecting the conservative nature of the updated hazard thresholds.
All 60 TTS outputs were successfully generated, with zero failures, demonstrating reliable end-to-end integration.

7.4.5. Latency & Real-time Performance

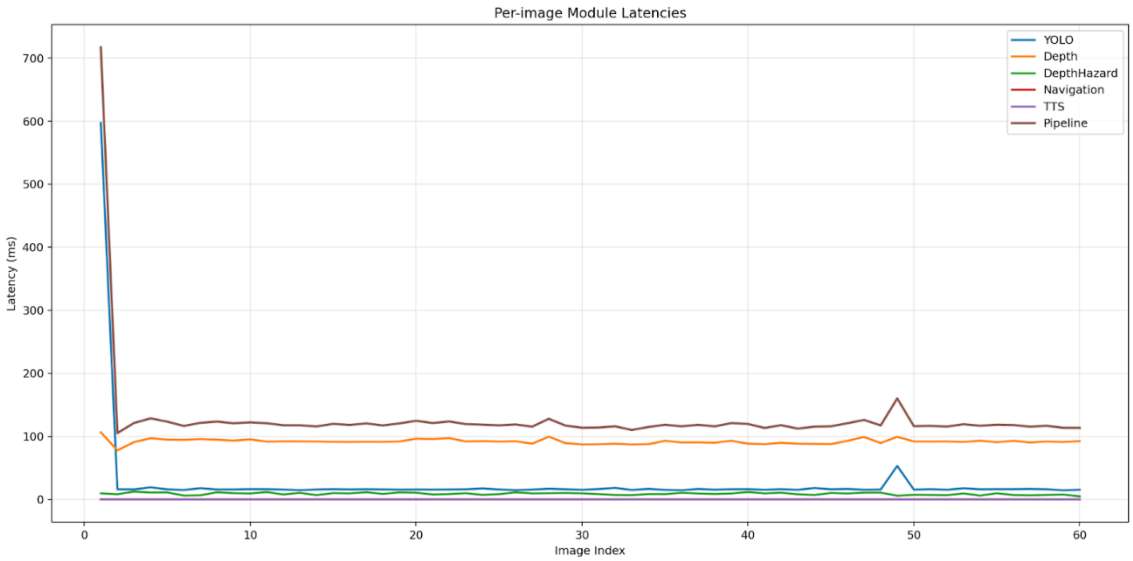
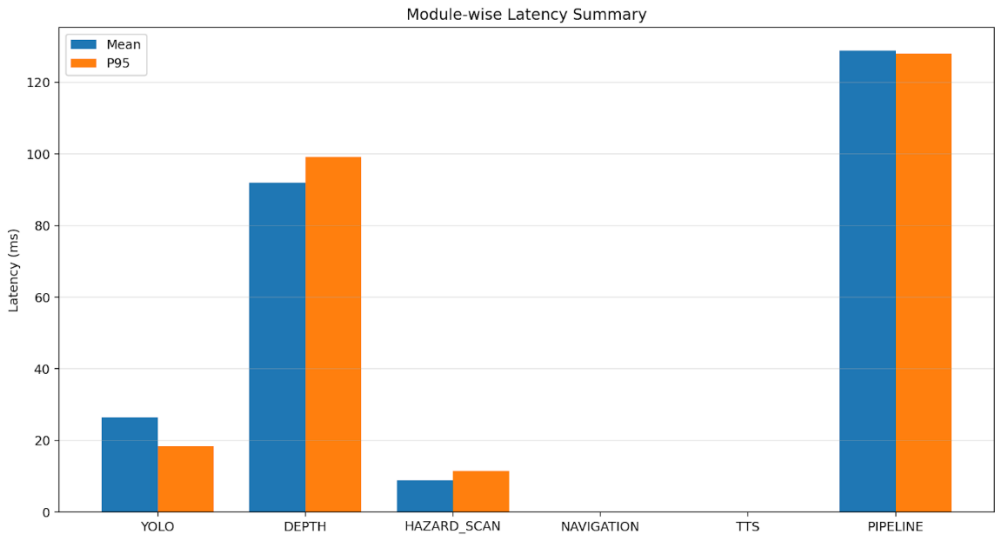


Figure 7.4.5.1 : Per-image latency breakdown across pipeline modules. Depth estimation dominates computational cost, while navigation and hazard scanning introduce negligible overhead.



The updated system shows a significant improvement in real-time performance. The total pipeline latency averages approximately 128 ms per frame, corresponding to an effective throughput of around 7–8 FPS. This represents a major improvement over earlier versions of the system.
Module-wise observations:

- Depth estimation (~92 ms) remains the dominant computational cost
- YOLO (~26 ms) operates efficiently
- Hazard scanning (~9 ms) and navigation (~0.02 ms) are negligible
- TTS latency (~0.003 ms) is effectively eliminated due to caching and optimization

These results indicate that the system is now capable of near real-time performance and is suitable for deployment on edge devices with moderate computational resources.

7.4.6. System-level Observations & Limitations

The end-to-end evaluation highlights several key system behaviors:

- The integration of depth-based hazard modeling significantly improves robustness, reducing dependence on object detection alone.
- The system exhibits conservative navigation behavior, prioritizing safety over aggressive movement.
- Despite high numerical depth errors, relative depth ordering remains sufficient for obstacle avoidance.

However, certain limitations persist:

- Object detection recall remains relatively low, leading to missed object classifications.
- Absolute depth estimation is unreliable due to scale ambiguity.
- The system tends to over-predict hazards, resulting in cautious navigation decisions.

7.4.7. Key Takeaway

The full system demonstrates that effective indoor navigation can be achieved through a combination of approximate perception and robust decision logic. By prioritizing safety through conservative hazard thresholds and leveraging relative depth information, the system maintains reliable performance even under imperfect detection and depth estimation.
This reinforces the principle that, for assistive navigation systems, consistent spatial reasoning and risk awareness are more critical than precise metric accuracy.

8. Module Integration and Pipeline Evaluation

Transforming isolated AI models into a functional, real-time assistive technology requires strict orchestration. This section outlines how the semantic (YOLO11s object detection), geometric (Depth-Anything-V2), and execution (Piper TTS) modules were fused into a unified application (app.py), followed by an aggressive empirical evaluation of the pipeline's overall throughput and precision on our SUN RGB-D dataset subset.

8.1. Pipeline Integration Architecture

To establish a cohesive perception-to-action flow, we implemented an advanced **Framebuffer Pipeline**. Instead of processing data streams arbitrarily, every visual frame acts as a unified state container passing through a highly controlled, synchronous layer:

1. **The Sensor Fusion Layer**
   Our pipeline enforces a strict precedence rule. When a frame enters the system:
   - **YOLO11s (Semantic):** Extracts 2D bounding boxes.
   - **Depth-Anything-V2 (Geometric):** Extracts a dense metric depth map.
   - **Spatial Alignment & Hazard Masking:** The bounding boxes are directly projected onto the depth matrix. To guarantee safety, we prioritize geometric limits over semantic boxes by applying aggressive threshold checks ($danger\_threshold\_m = 1.8m, warning\_threshold\_m = 2.5m$). Any depth pixels breaching the $1.8m$ danger threshold generate a masking penalty of 25.0 (compared to a standard bbox penalty).

2. **Asynchronous TTS Execution**
   To completely decouple the visual throughput overhead from the auditory engine, Piper Text-to-Speech runs asynchronously. Instead of pausing the frame loop to generate audio, the navigation string is *enqueued* directly. The application loop jumps instantly to the next frame, dropping the TTS pipeline execution cost to a nearly invisible fractional millisecond (measured at 0.0038 ms).

8.2. Pipeline Latency Benchmark (Hardware Execution Metrics)

During our final module evaluation, the fully integrated sequential pipeline processed 60 heavily cluttered validation images on CUDA. Microsecond-accurate milestones were recorded to evaluate edge-deployment viability.

Pipeline Stage	Mean Latency (ms)	P50 Latency / Median (ms)
YOLO11s Object Detection	26.36 ms	16.07 ms
Depth-Anything-V2 Estimation	91.92 ms	91.86 ms

Hazard Coordinate Scan	8.96 ms	9.43 ms
Navigation Logic Engine	0.025 ms	0.024 ms
Piper TTS (Async Enqueue)	0.0038 ms	0.0035 ms
TOTAL PIPELINE LATENCY	128.84 ms	117.87 ms

Note on Latency Results: By optimizing the process stream sequentially on GPU, the pipeline achieved an impressive 128.84 ms average total loop latency per frame (~ 7.7 FPS natively). Furthermore, the initial Model Load operations took just 6.14 seconds, and the TTS phrase pre-warming into RAM completed in 5.56 seconds seamlessly at boot.

8.3. Precision Evaluation against the SUN RGB-D Dataset

Beyond inference speed, an active navigation pipeline must demonstrate highly conservative decision-making within extremely cluttered spaces. The aforementioned 60-image evaluation batch purposefully extracted data from SUN RGB-D—a dataset characterized by dense living rooms, tight hallways, and severe occlusions.

8.3.1. Object Detection (YOLO11s) Module Metrics

Evaluating over the test frames (at an IOU threshold of 0.5 and confidence of 0.25), our fine-tuned YOLO11s model delivered the following metrics:

- Mean IOU: 0.865 (Demonstrating incredibly tight, accurate bounding box alignment on indoor hazards).
- Precision: 0.581
- Recall: 0.383 (F1 Score: 0.462)

8.3.2. Depth Estimation (Geometry) Metrics

The monocular depth map generated incredibly responsive masking to nearby obstacle geometry.

- **AbsRel Mean Error:** 0.535
- **Delta 1 Accuracy:** 0.516
- **Safety Trigger Consistency:** The system proved highly sensitive to nearby proximity threats. In 58 out of the 60 tested SUN RGB-D images, the pipeline identified hazardous pixels bleeding past the strictly conservative <1.8m limit. The mean depth of the absolute *nearest* hazards mapped in these dense rooms was an explicitly dangerous 1.098 meters.

8.3.3. Navigation Command Distribution

To prove that the pipeline interprets this data into actionable logic rather than freezing, we tracked the decisions synthesized by the navigation logic layer across the SUN RGB-D images. Out of the 60 images processed:

- "*Path blocked. Scan around.*": Issued 25 times.
- "*Turn left.*": Issued 14 times.
- "*Turn right.*": Issued 11 times.
- "*Move slightly right.*": Issued 5 times.
- "*Move slightly left.*": Issued 4 times.
- "*Go straight.*": Issued 1 time.

Analysis of Distribution: Because SUN RGB-D frames consistently feature dense, cluttered spaces with severe center-path obstacles, the navigation logic accurately identified that almost none of the forward routes were clear (yielding only 1 positive "Go straight" directive). Bouncing between *"Path blocked"* stops and sharp turns verifies that the mathematical depth threshold mappings heavily prioritize user safety and directly prevent forward collisions. Throughput 0 TTS failures over the 60 simulated calls.

8.3.4. Inferences

9. How to Use & Deployment

Given that our active blind-navigation assistant is designed as an edge-oriented, real-time feedback system, successful deployment requires stringent execution environments. This section covers how the application can be utilized locally, verified through our local UI interfaces, and deployed across external computational platforms, including mobile processors and artificial hardware constraints.

9.1. How to Use (Local Pipeline Setup)

To execute the core perception pipeline natively on a local machine, engineers and users must initialize the foundational CLI tools.

1. **Cloning the Repository**
   Begin by cloning the main repository to your local directory:

   ```
   git clone https://github.com/Saransh482003/Group-4-DS-and-AI-Lab-Project
   cd Group-4-DS-and-AI-Lab-Project
   ```

2. **Environment Configuration**
   Due to the vast matrix of deep-learning libraries, we rely on a strict virtual environment setup.

   ```
   python -m venv env
   source env/bin/activate  # On Windows: env\Scripts\activate
   pip install -r requirements.txt
   ```

3. **Model Weights Initialization**
   The local system requires access to the exact fine-tuned inference weights. Ensure the following files exist (download or map them from internal training logs):

   - YOLO11s-Final-Training.pt
   - depth_anything_v2_metric_hypersim_vits.pth
   - en_US-amy-medium.onnx (The Piper TTS engine)

4. **Triggering the Real-Time Pipeline**
   To launch the primary deterministic navigation logic bounding all three modules, trigger the main script wrapper from the terminal:

   ```
   python app/app.py
   ```

   What this does: Starting the script initiates an OpenCV camera capturing loop. The engine allocates the dual-stream models (YOLO11s and Depth-Anything-V2) to the active local hardware (CUDA or CPU), concurrently caching the Piper execution environment, and launching the dense visual window showcasing semantic boundaries masked by the geometric danger threshold overlays.

9.2. Running the Streamlit Web Application Locally

To facilitate easier testing of the pipeline via a graphical user interface (GUI) before pushing cloud artifacts, a local Streamlit web application was designed.

1. **Setup Required**
   Using the local UI requires specific cloud-oriented packages. These must be installed from the Hugging Face requirements list:

   ```
   pip install -r requirements-hf.txt
   ```

2. **Launching the App**
   To trigger the UI server natively on your machine, deploy the Streamlit command binding the local UI parameters:

   ```
   streamlit run app/streamlit_app.py
   ```

   This process automatically spans a localized port (typically ***http://localhost:8501***), generating an interactive web console mirroring our cloud-hosted application, entirely bypassing internet API throttling for rapid module debugging.

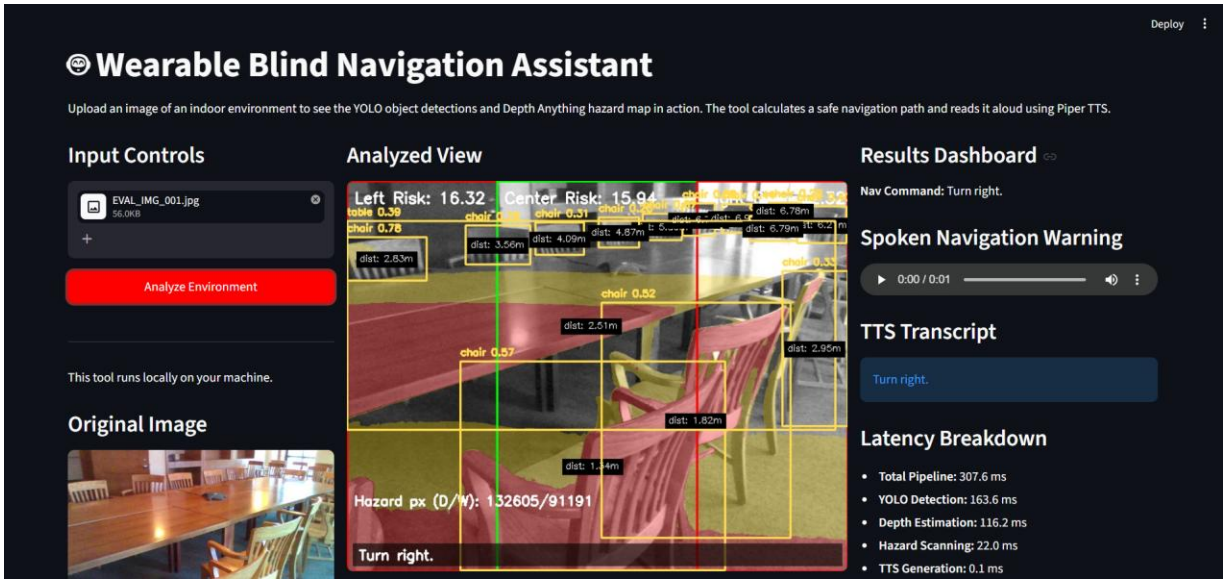
9.3. Deployment Implementations & Milestones

The final objective of this application is deployment. During this milestone, three distinct environments were targeted and actively configured.

9.3.1. Hugging Face Spaces Deployment (Cloud Testing Environment)

To make the application universally accessible for testing evaluations by developers and external contributors, we established a Hugging Face Space (***Saransh482003/DSAI-Project-Navigation-Assistant***).

- **How it Started:** Instead of pushing the repository blindly, a programmatic `deploy.py` script was engineered using the `HfApi()`.
- **Setup Logistics:** The deployment strictly uploads target directories via `api.upload_folder()`, circumventing heavy, unnecessary background tracking files. The destination server boots directly from a custom `Dockerfile` based on python:3.10-slim. It invokes `apt-get` logic to explicitly grant vital Linux dependencies (libgl1 and libglib2.0-0) that are natively absent but critically enforced by headless OpenCV bindings.
- By funnelling through Hugging Face's Git LFS protocol, the large $.pt$ and $.pth$ weight architectures load directly into the container seamlessly.



9.3.2. Mobile Edge-Deployment Setup (Android Implementation)

Because visually impaired individuals require mobility, we initialized the architectural transition to Android application deployment.

- **The ONNX Vectorization:** Android ARM processors cannot reliably buffer massive $.pt$ PyTorch tensors at high FPS. Thus, the script `export_android_models.py` was developed specifically to translate the native models. The script actively strips PyTorch wrappers and forcefully cross-compiles both the ***YOLO11s object detector*** and the ***ViT-S Depth-Anything-V2*** architectures natively into standard `.onnx` formats (locking explicitly with `opset=14` for cross-platform determinism). Seeing as Piper TTS inherently utilizes ONNX, this breakthrough aligns all three modules into standard mobile-executable footprints.

9.3.3. Simulating Raspberry Pi 5 Wearable Environments via Docker

The ultimate vision for this navigation architecture involves a head-mounted wearable device running completely locally off a Raspberry Pi 5. Since we did not have physical access to the Pi 5 hardware throughout the duration of this milestone testing phase, we leveraged Docker constraints to mimic it.

- **The Approach:** We engineered a local Docker Container artificially bottlenecked to reflect the physical parameter ceilings of a Raspberry Pi 5 (8GB RAM boundaries, heavily throttled computing constraints tracking ARM behaviors).
- **The Result:** Pushing the navigation pipeline directly inside this simulated bottleneck heavily dictated our earlier evaluations. Because the simulated hardware began instantly dropping frames beneath real-time limits, it catalyzed the invention of our Pipeline's **"Process Every N Frames" caching heuristic** and the eventual removal of our failing Parallel Execution mechanism. Evaluating against this constrained container mathematically secures that our deterministic logic engine guarantees pipeline functionality once physically burned onto a commercial Raspberry Pi later down the line.

10. Member Testification

Name	Email	Testification
Saransh Saini	22f1001123@ds.study.iitm.ac.in	I testify
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	I testify
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	I testify
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	I testify
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	I testify